Moduły i argumenty wiersza polecenia

Po co dzielić program?

Program pobierający dwie liczby i wyświetlający ich sumę może mieć kilka wierszy. W takim programie dodatkowe funkcje nie muszą ułatwiać czytania. Przy kilkuset wierszach sytuacja wygląda inaczej: trudniej odnaleźć obliczenia, zmienić jedną część i sprawdzić ją niezależnie od pozostałych.

Funkcje pozwalają nazwać poszczególne czynności i oddzielić je od siebie, nawet gdy żadna nie jest wykonywana dwa razy. Podobnie moduły pozwalają podzielić większy program na pliki odpowiadające za różne jego elementy. Korzyść z tego podziału jest podobna do korzyści z dzielenia programu na funkcje — ułatwia organizację większego programu.

Druga korzyść to możliwość ponownego użycia kodu. Klasy i funkcje z jednego modułu mogą służyć wielu programom bez kopiowania ich definicji.

Zwykły plik jako moduł

Moduł nie wymaga specjalnego oznaczenia w kodzie. Zwykły plik źródłowy Pythona można wykorzystać jako moduł. Import wykonuje jego kod, w tym definicje klas i funkcji. Kilka różnic między importem a bezpośrednim uruchomieniem jest opisanych dalej.

Python może szukać modułów w różnych miejscach, zależnie od ustawień środowiska, ale najprościej umieścić własny moduł w tym samym katalogu co korzystający z niego program.

Zacznijmy od programu `trasy_program.py`. Zawiera znane klasy oraz krótką demonstrację tworzenia użytkownika z trasą.

Plik `trasy_program.py`:

```
"""Klasy wspólne dla programów pracujących z trasami."""

import copy


class Wspolrzedne:
    """Współrzędne punktu: długość i szerokość geograficzna."""

    def __init__(self, dlugosc, szerokosc):
        self.dlugosc = dlugosc
```

```
        self.szerokosc = szerokosc

    def __repr__(self):
        return (
            f"Wspolrzedne({self.dlugosc!r}, {self.szerokosc!r})"
        )


class Trasa:
    """Nazwana trasa złożona z kolejnych punktów."""

    def __init__(self, nazwa):
        self.nazwa = nazwa
        self.przebieg = []

    def dodaj_punkt(self, punkt):
        """Dodaj punkt na końcu trasy."""
        self.przebieg.append(punkt)

    def __repr__(self):
        return (
            f"<Trasa nazwa={self.nazwa!r}, "
            f"przebieg={self.przebieg!r}>"
        )


class Uzytkownik:
    """Użytkownik z listą ulubionych tras."""

    def __init__(self, imie, nazwisko):
        self.imie = imie
        self.nazwisko = nazwisko
        self.ulubione_trasy = []

    def dodaj_ulubiona(self, trasa):
        """Dodaje ulubioną trasę."""
        self.ulubione_trasy.append(trasa)

    def __repr__(self):
        return (
            f"<Uzytkownik imie={self.imie!r}, "
            f"nazwisko={self.nazwisko!r}, "
            f"ulubione_trasy={self.ulubione_trasy!r}>"
        )

    def kopiuj_ulubione(self, inny):
        """Kopiuje ulubione trasy innego użytkownika."""
        self.ulubione_trasy.extend(
            copy.deepcopy(inny.ulubione_trasy)
        )
```

```
def main():
    punkt1 = Wspolrzedne(22.5, 51.2)
    punkt2 = Wspolrzedne(22.6, 51.3)
    trasa = Trasa("Spacer")
    trasa.dodaj_punkt(punkt1)
    trasa.dodaj_punkt(punkt2)
    jan = Uzytkownik("Jan", "Kowalski")
    jan.dodaj_ulubiona(trasa)
    print(jan)


main()
```

W notebooku znak `!` na początku komórki pozwala uruchomić polecenie, np. osobny program. W poniższych przykładach używamy polecenia `python`. Jeśli w Twoim środowisku interpreter uruchamia się innym poleceniem, np. `python3` lub `py`, odpowiednio je zastąp. Interpreter uruchomiony poleceniem powinien należeć do środowiska, w którym chcesz sprawdzać przykłady.

In [2]:
```
!python trasy_program.py
```

```
<Uzytkownik imie='Jan', nazwisko='Kowalski', ulubione_trasy=[<Trasa nazwa='S
pacer', przebieg=[Wspolrzedne(22.5, 51.2), Wspolrzedne(22.6, 51.3)]>]>
```

W programie `program1.py` chcemy użyć tych klas, skopiować trasę i zmienić kopię. Nie potrzebujemy ponownie uruchamiać demonstracji użytkownika.

Plik `program1.py`:

```
import copy
import trasy_program


def main():
    trasa = trasy_program.Trasa("Spacer")
    trasa.dodaj_punkt(trasy_program.Wspolrzedne(22.5, 51.2))
    trasa.dodaj_punkt(trasy_program.Wspolrzedne(22.6, 51.3))
    kopia = copy.deepcopy(trasa)
    kopia.nazwa = "Dłuższy spacer"
    kopia.przebieg[1].szerokosc = 51.4
    print(trasa)
    print(kopia)


if __name__ == "__main__":
    main()
```

In [3]:
```
!python program1.py
```

```
<Uzytkownik imie='Jan', nazwisko='Kowalski', ulubione_trasy=[<Trasa nazwa='S
pacer', przebieg=[Wspolrzedne(22.5, 51.2), Wspolrzedne(22.6, 51.3)]>]>
<Trasa nazwa='Spacer', przebieg=[Wspolrzedne(22.5, 51.2), Wspolrzedne(22.6,
51.3)]>
<Trasa nazwa='Dłuższy spacer', przebieg=[Wspolrzedne(22.5, 51.2), Wspolrzedn
e(22.6, 51.4)]>
```

Przed dwiema trasami pojawił się jednak użytkownik z demonstracji. Import wykonał cały kod pliku `trasy_program.py`, włącznie z bezwarunkowym wywołaniem funkcji `main`.

Kod wykonywany tylko przy bezpośrednim uruchomieniu

Zmienna `__name__` pozwala rozróżnić oba przypadki. Przy bezpośrednim uruchomieniu pliku ma wartość `"__main__"`, a przy imporcie pliku `trasy.py` ma w nim wartość `"trasy"`.

W pliku `trasy.py` pozostawiamy te same definicje. Zmieniamy tylko końcowe wywołanie:

```
if __name__ == "__main__":
    main()
```

Nazwa `main` jest konwencją. Zwykle nazywa się tak funkcję od której program rozpoczyna działanie, ale Python nie wymaga tej nazwy.

Przy bezpośrednim uruchomieniu `trasy.py` demonstracja nadal działa:

In [4]:
```
!python trasy.py
```

```
<Uzytkownik imie='Jan', nazwisko='Kowalski', ulubione_trasy=[<Trasa nazwa='S
pacer', przebieg=[Wspolrzedne(22.5, 51.2), Wspolrzedne(22.6, 51.3)]>]>
```

Program `program2.py` różni się od `program1.py` wyłącznie użyciem modułu `trasy` zamiast `trasy_program`:

Plik `program2.py`:

```
import copy
import trasy


def main():
    trasa = trasy.Trasa("Spacer")
    trasa.dodaj_punkt(trasy.Wspolrzedne(22.5, 51.2))
    trasa.dodaj_punkt(trasy.Wspolrzedne(22.6, 51.3))
    kopia = copy.deepcopy(trasa)
    kopia.nazwa = "Dłuższy spacer"
    kopia.przebieg[1].szerokosc = 51.4
```

```
    print(trasa)
    print(kopia)


if __name__ == "__main__":
    main()
```

In [5]:
```
!python program2.py
```

```
<Trasa nazwa='Spacer', przebieg=[Wspolrzedne(22.5, 51.2), Wspolrzedne(22.6,
51.3)]>
<Trasa nazwa='Dłuższy spacer', przebieg=[Wspolrzedne(22.5, 51.2), Wspolrzedn
e(22.6, 51.4)]>
```

Teraz widzimy tylko trasy tworzone w programie. Import nadal wykonał definicje i sprawdzenie warunku, ale nie wywołał funkcji `main` modułu.

Mechanizm importu zapamiętuje już zaimportowany moduł. Poniższa komórka powoduje tylko jednokrotne uruchomienie modułu (i jednokrotne wyświetlenie użytkownika):

In [6]:
```
import trasy_program
import trasy_program
```

```
<Uzytkownik imie='Jan', nazwisko='Kowalski', ulubione_trasy=[<Trasa nazwa='S
pacer', przebieg=[Wspolrzedne(22.5, 51.2), Wspolrzedne(22.6, 51.3)]>]>
```

Jeżeli uruchomisz tę samą komórkę ponownie, to nie wyświetli już niczego. Moduł był już wcześniej zaimportowany i kod modułu nie będzie już w ogóle wykonany.

Sposoby importowania

Sposoby importowania własnego modułu działają tak samo jak przy modułach biblioteki standardowej. Poniższe programy uruchamiamy osobno, aby nazwy po wcześniejszych importach nie wpływały na działanie następnego przykładu.

Plik `przyklad_import_1.py`:

```
import trasy

print(trasy.Wspolrzedne(1.5, 2.5))
print(trasy.Trasa("spacer"))
```

In [7]:
```
!python przyklad_import_1.py
```

```
Wspolrzedne(1.5, 2.5)
<Trasa nazwa='spacer', przebieg=[]>
```

Plik `przyklad_import_2.py`:

```
import trasy as t
```

```
print(t.Wspolrzedne(1.5, 2.5))
print(t.Trasa("spacer"))
```

In [8]:
```
!python przyklad_import_2.py
```

```
Wspolrzedne(1.5, 2.5)
<Trasa nazwa='spacer', przebieg=[]>
```

Plik `przyklad_import_3.py`:

```
from trasy import Wspolrzedne

print(Wspolrzedne(1.5, 2.5))
print(Trasa("spacer"))
```

Ten program zaimportuje tylko nazwę `Wspolrzedne`. Utworzenie punktu się powiedzie, ale próba użycia nieudostępnionej nazwy `Trasa` zakończy program wyjątkiem `NameError`.

In [9]:
```
!python przyklad_import_3.py
```

```
Wspolrzedne(1.5, 2.5)
Traceback (most recent call last):
  File "/tmp/zpwjp-cz3-y49ydhfh/przyklad_import_3.py", line 4, in <module>
    print(Trasa("spacer"))
          ^^^^^
NameError: name 'Trasa' is not defined
```

Plik `przyklad_import_4.py`:

```
from trasy import Wspolrzedne as W

print(W(1.5, 2.5))
```

In [10]:
```
!python przyklad_import_4.py
```

```
Wspolrzedne(1.5, 2.5)
```

Także wybiórczy import (np. `from trasy import Wspolrzedne`) wykonuje cały kod modułu przy jego pierwszym imporcie. Wybór nazw po słowie `import` określa nazwy dostępne w korzystającym z niego programie, a nie fragment kodu do wykonania.

Argumenty wiersza polecenia

Program może otrzymać pewne dane od razu przy uruchomieniu. **Argumenty wiersza polecenia** (ang. *command-line arguments*) zapisujemy po nazwie programu przy uruchomieniu go w wierszu polecenia (konsoli, terminalu). Argumenty te dostępne są jako zmienna `sys.argv`. Jest to lista napisów: pierwszy element zawiera nazwę skryptu, pozostałe — jego argumenty. Listę samych argumentów (bez nazwy skryptu) możemy uzyskać np. za pomocą

`sys.argv[1:]`, a liczbę przekazanych argumentów możemy uzyskać np. za pomocą `len(sys.argv) - 1`.

Program `pokaz_plik.py` korzysta z pierwszego argumentu, a jeżeli go nie ma, pyta o nazwę pliku:

Plik `pokaz_plik.py`:

```
import sys


def wyswietl_plik(nazwa):
    with open(nazwa, encoding="utf-8") as plik:
        print(plik.read(), end="")


def main():
    if len(sys.argv) > 1:
        nazwa = sys.argv[1]
    else:
        nazwa = input("Nazwa pliku: ")
    wyswietl_plik(nazwa)


if __name__ == "__main__":
    main()
```

Najpierw wypróbujemy uruchomienie bez argumentu. Wypróbujmy działanie na plikach `zdanie.txt` i `opis zdania.txt`. Używamy polecenia IPython `%run`, które pozwala uruchamiać w notebooku programy interaktywne. Skrypt uruchomiony przez `!python` zwykle nie ma w notebooku możliwości pracy interaktywnej. W terminalu lub IDE można uruchomić program bez argumentu w zwykły sposób.

In [11]:
```
%run pokaz_plik.py
```

```
Nazwa pliku: zdanie.txt
Zażółć gęślą jaźń
```

Z argumentem nie trzeba odpowiadać na pytanie:

In [12]:
```
!python pokaz_plik.py zdanie.txt
```

```
Zażółć gęślą jaźń
```

W powyższym wywołaniu zmienna `sys.argv` będzie miała wartość `['pokaz_plik.py', 'zdanie.txt']`. Jeżeli nazwa zawiera spację, możemy otoczyć ją cudzysłowem, żeby była potraktowana jako jednen argument:

In [13]:
```
!python pokaz_plik.py "opis zdania.txt"
```

```
„Zażółć gęślą jaźń” to krótkie, poprawne gramatycznie zdanie
zawierające wszystkie polskie litery ze znakami diakrytycznymi.
Zdanie nie ma szczególnie sensownego znaczenia.
Zdanie bywa używane do testowania kodowania i wyświetlania polskich znaków.
```

Wiele argumentów

W programie `suma.py` wszystkie argumenty są liczbami do dodania. Tym razem nie ma wariantu interaktywnego. Argumenty są napisami, dlatego zamieniamy je na liczby przez `float`.

Plik `suma.py`:

```
import sys


def main():
    suma = 0.0
    try:
        for argument in sys.argv[1:]:
            suma += float(argument)
    except ValueError:
        print("Argumenty muszą być liczbami.")
    else:
        print(suma)


if __name__ == "__main__":
    main()
```

In [14]:
```
!python suma.py 1.5 2 -0.5
```

```
3.0
```

In [15]:
```
!python suma.py 1.5 tekst
```

```
Argumenty muszą być liczbami.
```

Dla niepoprawnego argumentu program wypisuje informację zamiast częściowej sumy. Bez argumentów suma wynosi `0.0`.

Uruchamianie w IDE

Przycisk uruchamiania w IDE także uruchamia interpreter z argumentami, a ich wartości możemy odpowiednio skonfigurować:

- W PyCharm użyj pola **Parameters** w **Run | Edit Configurations**, np. `zdanie.txt`, przy skrypcie `pokaz_plik.py`. Dokumentacja PyCharm.
- W VS Code odpowiednikiem jest np. `"args": ["zdanie.txt"]` w konfiguracji uruchamiania/debugowania. Dokumentacja VS Code.

Czy programy bez graficznego interfejsu użytkownika są przestarzałe?

Programy z graficznym interfejsem użytkownika (ang. *graphical user interface*, GUI) wydają się dużo wygodniejsze w użyciu niż programy przeznaczone do użytku w wierszu poleceń (ang. *command-line interface*, CLI). Ale to dotyczy konkretnego przypadku użycia i zamierzonego zastosowania programu.

Program sterowany argumentami wiersza polecenia można wielokrotnie uruchamiać dla różnych danych, bez ręcznego odpowiadania na pytania, czy „wyklikiwania" opdowiednich opcj i danych. Łatwiej włączyć go do większego procesu przetwarzania i połączyć z innymi narzędziami. Raz przygotowane polecenie może służyć do automatycznego przetworzenia wielu plików. W przypadku narzędzi sterowanych przez GUI taka automatyzacja i integracja jest znacznie trudniejsza.

Nieprzypadkowo wiele podstawowych narzędzi programistycznych jest projektowana przede wszystkim do obsługi z wiersza poleceń. Dotyczy to między innymi kompilatorów, interpreterów języków programowania, systemów kontroli wersji (np. Git) czy narzędzi do konteneryzacji (np. Docker). Dla wielu z nich dostępne są również interfejsy graficzne, ale zwykle stanowią one dodatkową warstwę ułatwiającą korzystanie z narzędzia. Interfejs graficzny jest podstawowym sposobem obsługi przede wszystkim tam, gdzie ma to szczególne uzasadnienie, jak w przypadku zintegrowanych środowisk programistycznych (ang. *integrated development environment*, IDE).

Dobrym przykładem jest sam interpreter Pythona. Można uruchamiać go bezpośrednio z terminala, przekazując mu odpowiednie argumenty, ale można też korzystać z niego za pośrednictwem IDE. Pierwszy sposób szczególnie dobrze nadaje się do automatyzacji i łączenia programów w większe procesy przetwarzania, drugi ułatwia interaktywną pracę nad kodem. Oba sposoby korzystają z tego samego interpretera, ale odpowiadają na różne potrzeby.